3D Snake

Grupp Pilgrimsfalkarna

Kravdokument

V. 1.3

2025-05-28

Innehåll

Dokumenthistorik 3
Kravdokument 4
Syfte 4
Produktbeskrivning 4
Målgrupp 5
Ordlista 6
Referenser 6
Funktionella krav 7
Prioritering för kraven. 7
Beskrivning av id:t för funktionella krav 7
Server 7
Kommunikation 7
Spellogik för Snake 8
Felsökning 9
Highscore 9
Felsökning 10
Klient 10
Kommunikation 10
GUI 10
Inbyggda system 11
Kommunikation 11
Solenoidstyrning 12
LED-styrning 12
Kvalitativa krav 12
Prioritering för kraven. 12
Beskrivning av id:t för kvalitativa krav 12
Java-projekt 12
Programmering 13
Systemet som helhet 13
Inbyggda system 14

Dokumenthistorik

Datum	Version	Beskrivning	Författare
250416	1.0	Skrivit text för syfte, beskrivning, målgrupp, intressenter samt börjat lägga till olika typer av krav	Hela gruppen
250510	1.1	Skrev om vissa krav så att de blev mer förståeliga och de krav utan beskrivning fick en enkel beskrivning	Burak
250521	1.1	Kommenterat vissa krav så att den eller de som är insatta i det kan kolla och förändra/förbättra	Burak
250522	1.2	Lagt till nya funktionella och kvalitativa krav. Formaterat vissa krav som var otydliga och gjort en bättre gruppering av kraven.	Burak & Annika & Othman
250528	1.3	Formulerat om ett krav för att göra det tydligare och delar in i kategorier	Burak & Annika & Othman

Kravdokument

Syfte

Syftet med detta dokument är att specificera kraven för utveckling av ett interaktivt Snake-spel med tredimensionell fysisk komponent som rör sig upp och ner på spelbrädan. Projektet kombinerar spelutveckling med hårdvarunära programmering och utgör en tillämpning av systemutveckling i en kontext där klient, server och inbyggda system samverkar.

Dokumentet syftar till att fungera som grund för hur projektgruppen ska planera, utveckla och testa produkten. Syftet är även att säkerställa att alla systemdelar uppfyller krav enligt kursens direktiv om integrering av mjukvara och hårdvara.

Dokumentet innehåller en utförligare beskrivning av produkten, dess användare och intressenter. En kravspecifikation presenteras där funktionella och icke-funktionella krav är uppdelade.

Projektet syftar också till att ge en lärorik insikt i samverkan mellan realtidskommunikation, spelmekanik och fysiskt gränssnitt, samt skapa en produkt som både är tekniskt utmanande och visuellt engagerande.

Produktbeskrivning

Produkten är en spelapplikation som bygger på det klassiska spelet Snake men med en unik tredimensionell twist som inte har skådats förut, där Ormens rörelse visualiseras genom att en kub lyfts upp från spelbrädan i realtid. Detta ger en tydlig koppling mellan mjukvaran och den verkligen världen samt att det ger spelaren en fysisk återkoppling vilket många tycker är engagerande.

Spelet styrs genom en fysisk kontrollpanel kopplat till det inbyggda systemet som registrerar användarens input genom knappar och skickar sedan vidare detta till servern. Servern innehåller all spellogik och hanterar bland annat ormens position, kollisioner, poäng osv. Denna ansvarar även för att kommunicera tillbaka till det inbyggda systemet där det specificeras vilken ruta som ska lyftas. Denna signal skickas även till klienten som representerar spelplanen visuellt på en datorskärm.

Den visuella återkopplingen sker via en fysisk spelbräda där aktiva rutor lyfts upp med hjälp av en egenutvecklad solenoid. Detta ger en arkadliknande upplevelse som både är interaktiv och engagerande.

Systemet är uppdelat i tre huvudsakliga komponenter

- En **server** som hanterar spelets logik och kommunikation mellan komponenterna
- Ett **inbyggt system** som ansvara för fysisk återkoppling samt input på knappar
    - Det inbyggda systemet agerar även som en klient som kopplar upp sig till servern

- En **klient** som visualiserar spelet på en datorskärm och visar en highscore lista med de bästa spelarna

Målgrupp

Projektet riktar sig främst mot individer som är intresserade av teknik, spel och fysiska interaktiva interaktioner. Målgruppen inkluderar elever, gamers och även personer som är nyfikna på hur "gamla" digitala spel kan visualiseras fysiskt med en modern twist där digitala och fysiska element kombineras.

Spelet är tänkt att demonstreras i miljöer där teknisk kreativitet premieras, exempelvis på teknikmässor, skolpresentationer, makerspaces eller spelrelaterade evenemang. Det lämpar sig väl som ett pedagogiskt exempel på integration mellan mjukvara och hårdvara, och kan därför även användas som lärresurs i utbildningar inom teknik, elektronik eller programmering

Ordlista

Solenoid	Elektromagnetisk aktuator som lyfter/sänker kuber på spelplanen.
Half-duplex	Både parter i kommunikationen kan skicka data genom samma anslutning, men inte samtidigt
Multiplexing	Möjliggör att flera signaler överförs genom en enda kanal.
CamelCase	Vid namngivning med flera ord skrivs alla ord förutom det första med stor bokstav i början. Ex camelCase
PascalCase	Vid namngivning med flera ord skrivs alla ord med stor bokstav i början. Ex PascalCase

Referenser

[1] Agile Business Consortium, “MoSCoW Prioritisation,” *Agile Business Consortium*. [Online]. Tillgänglig: https://www.agilebusiness.org/dsdm-project-framework/moscow-prioririsation.html [Hämtad: 26 maj 2025].

[2] Google, “Java,” *Oracle Java Code Conventions*. [Online]. Tillgänglig: https://google.github.io/styleguide/javaguide.html [Hämtad: 27 maj 2025].

[3] Google, “C++ Version,” *Google C++ Style Guide*. [Online]. Tillgänglig: https://google.github.io/styleguide/cppguide.html [Hämtad: 26 maj 2025].

Funktionella krav

De funktionella kraven beskriver systemets funktioner och beteende. De förklarar vad systemet ska göra. Kraven är indelade i 3 delar; server, klient och inbyggda system.

Prioritering för kraven.

De funktionella kraven prioriteras enligt MoSCoW[1]-modellen.

- **Must**: Kravet måste implementeras för att produkten ska fylla sitt syfte.
- **Should**: Kravet ska implementeras för att slutanvändaren ska ha en bättre spelupplevelse.
- **Could**: Kravet kan implementeras för att om det finns tid som en utbyggnad.
- **Won't**: Kravet ska inte implementeras för att det inte fyller ett syfte till slutprodukten eller att det inte finns tid över för att implementeras.

Beskrivning av id:t för funktionella krav

Första bokstaven är F och står för att det är ett funktionellt krav. De kommande bokstäverna är för vilket område kravet tillhör och sista siffran för att numrera så de blir unika.
Exempelvis F.S.K.1 = Funktionellt krav inom server-kommunikationen med nummer 1.

[Krav ska vara testbart formulerade.

Generellt för hela dokumentet: Om det anses göra texten tydligare går det att dela in text i olika delar med underrubriker. Det är inte nödvändigt att använda underrubriker och då följer texten direkt efter huvudrubriken utan underrubrik.]

Server

Kommunikation

F.S.K.1 Kommunikation med klient (Must)

Servern initierar och upprätthåller en tvåvägskommunikation med klienten.

F.S.K.1.1 TCP socket (Must)

Kommunikationen med klienten sker med en TCP socket

F.S.K.1.2 Permanent anslutning med klienten (Should)

Anslutningen med klienten är alltid öppen oavsett om spelets tillstånd. Om spelet avslutat stängs anslutningen inte, utan hamnar i vänteläge.

F.S.K.1.3 Dataöverföring (Must)

Server skickar uppdateringar till klienten som uppdaterar gui:t.

[1] MoSCoW model

F.S.K.2 Kommunikation med Arduino (Must)
Servern initierar och upprätthåller en tvåvägskommunikation med Arduinon.

F.S.K.2.1 TCP/IP kommunikation (Must)
Kommunikationen mellan servern och arduinon sker via en TCP socket

F.S.K.2.2 Dataöverföring (Must)
Servern tar emot knapptryck från inbyggda systemet och uppdaterna ormens position i spellogiken

F.S.K.2.3 Half-Duplex (Must)
Data skickas från och till Arduinon och servern genom en half-duplex kommunikation.
Arduinon används för både knappläsning och för att styra solenoiederna men körs på en kärna, vilket gör att det är en half-duplex med snabba byten.

Spellogik för Snake
F.SS.S.1 Spelregler för Snake (Must)
Spelet följer de generella reglerna för Snake spelet.

F.SS.S.1.1 Start game (Should)
Klienten frågar om att starta spelet.

F.SS.S.1.2 Game over (Must)
Spelet är slut när ormen krockar in i väggen eller går in i sig själv.

F.SS.S.1.3 Vinst (Must)
Det ska inte vara möjligt att "vinna", spelare kan endast samla poäng och de med högst poäng läggs till i en highscore-lista

F.SS.S.1.3 Äpplet förflyttas (Should)
Äpplet förflyttas efter 30 sekunder om den inte ätits av ormen.
Detta krav behöver implementeras för att solenoiderna blir överhettade om de är upp för länge.

F.SS.S.2 Förflyttning (Must)
Servern tar emot riktningskommandon från det inbyggda systemet för att sedan uppdatera logiken.

F.SS.S.2.1 Ej möjliga val (Must)
Det ska inte gå att styra ormen till den motsatta riktningen som den rör sig på i det aktuella läget

F.SS.S.3 Poängsystem (Should)
Varje äpple ger +1 poäng. Poäng visas i klientens GUI.

F.S.S.3.1 Hastighet (Won't)
Hastigheten påverkar antalet poäng som ges.

Felsökning
F.S.F.1 GUI (Could)
En panel på serversidan som visar spelplanen.
Denna panel är endast till för felsökning och kommer inte visas för användaren.

Highscore
F.S.H.1 En textfil (Should)
Highscore sparas som en textfil med namn och poäng.

F.S.H.1.1 ID (Should)
Ett id som är unikt för varje spelare.

F.S.H.2 Skickar informationen till klienten (Should)
Server skickar highscore listan till klienten.

Spellogik för ConnectFour
F.SC.S.1 Spelregler för ConnectFour(Must)
Spelet följer de generella reglerna för connect four spelet.

F.SC.S.1.1 Start game (Should)
Klienten frågar om att starta spelet.

F.SC.S.1.2 Game over (Must)
Spelet är slut när en spelare eller bot får 4 av sina brickor i rad. Antingen i horisontell, vertikal, eller diagonal riktning.

F.SC.S.1.3 Vinst (Must)
Det ska vara möjligt att vinna och det ska hända när ovanstående krav har uppfyllts med dina egna brickor.

F.SC.S.2 Placering (Must)
Servern tar emot placeringskommandon från det inbyggda systemet för att sedan uppdaterar logiken.

F.SC.S.2.1 Ej möjliga val (Must)
Om en kolumn är full så ska en spelare inte kunna släppa sin bricka där.

F.SC.S.3 Animation

F.SC.S.3.1 Fall-animation(should)
När en spelare släpper sin bricka så ska en "fall-animation" dyka upp där det ser ut som att spelarens bricka faller mot botten

F.SC.S.3.1 Vinst/förlust animation(should)
När en spelare vinner eller förlorar och spelet tar slut så ska alla brickor bli till motsatt färg av den som vann.

Felsökning
F.S.F.1 GUI (Could)
En panel på serversidan som visar spelplanen.
Denna panel är endast till för felsökning och kommer inte visas för användaren.

Klient

Kommunikation
F.K.K.1 Kommunikation med server (Must)
Klienten ska kunna ansluta sig till servern.

F.K.K.1.1 TCP socket (Must)
Kommunikation med servern sker med en TCP socket.

F.K.K.1.2 Återanslutning (Should)
Vid förlorad koppling mellan klient och server ska klienten försöka återansluta till server fram tills att en lyckad anslutning upprättas.

F.K.K.1.3 Dataöverföring (Must)
Klienten skickar olika kommandon till servern som representerar input från användaren

GUI
F.K.G.1 En panel med spelplanen (Must)
En panel som visar hur spelet ser ut. Motsvarar 3d-spelplanen.

F.K.G.1.1 Highscore lista visas live (Could)
En highscore lista finns på panelen och uppdateras så att spelaren kan jämföra sina poäng med tidigare spel.

F.K.G.1.2 Avsluta-knapp (Could)

Det finns en avsluta-knapp som gör att man kan stänga av spelet och gå tillbaka till spelmenyn

F.K.G.2 En startmeny med valmöjligheter (Could)
En meny som visar olika spel som användaren kan välja mellan.

F.K.G.2.1 Logga med olika spel (Could)
Det finns en unik spellogga för varje spel man kan välja att spela

F.K.G.3 En panel (=spelmeny) med info om det valda spelet (Should)
En liten textruta som ger användaren information om det valda spelet i form av kort text med enklar regler

F.K.G.3.1 Leaderboard lista (Should)
En lista med alla highscores där namn och poäng visas för spelet.

F.K.G.4 Namninput (Should)
En inputruta där man kan skriva in sitt namn för att hålla reda på highscore

F.K.G.4.1 Felhantering (Should)
Om användaren inte skriver in ett namn eller lämnar rutan blank så ska systemet komplettera med ett defaultname.

F.K.G.5 Startknapp (Should)
En knapp för att sätta igång spelet.

F.K.G.6. Tillbaka-knapp (Could)
En tillbaka-knapp som för en tillbaka till menyn med spelval

F.K.G.4 En panel vid Game Over (Won't)
En panel som dyker upp när spelaren förlorat.

Inbyggda system

Kommunikation
F.IS.K.1 Kommunikation med server (Must)
Systemet kommunicerar med java-servern.

F.IS.K.1.1 Ethernet (Flytta till kvalitativa)
Systemet skickar input från spelaren till servern via Ethernet

F.IS.K.1.1 TCP-socket (Must)
Systemet skickar data mellan servern genom en TCP-socket.

Solenoidstyrning

F.IS.S.1 Knappstyrning (Must)
Inbyggda systemet läser knapptryck (t.ex. Piltangenter, joystick eller knappar) och översätter detta till kommandon (ett tal från 1 till 4) som skickas till servern.

F.IS.S.3 Ta emot kommando (Must)
Systemet tar emot kommando från servern om vilken ruta som ska lyftas.

F.IS.S.4 Aktivera lyftmekanik (Must)
Systemet styr solenoider för att lyfta rätt ruta enligt serverns instruktioner

F.IS.S.5 Felhantering (Should)
Systemet upptäcker och hanterar utebliven kontakt med servern t.ex. via felmeddelande eller återkoppling

F.IS.S.6 Multiplexing (Must)
Inbyggda systemet använder multiplexing för att kontrollera flera solenoider med så få arduino outputs som möjligt.

LED-styrning

F.IS.L.1 Ta emot kommando (Must)
Systemet tar emot kommando från servern om vilken LED som ska tändas.

F.IS.L.2 Styra färg (Must)
Systemet styr LED matrix för att tända rätt färg enligt serverns instruktioner.

Kvalitativa krav

Prioritering för kraven.

De kvalitativa kraven prioriteras enligt MoSCoW [1] -modellen.
Must: Kravet måste implementeras för att produkten ska fylla sitt syfte.
Should: Kravet ska implementeras för att slutanvändaren ska ha en bättre spelupplevelse.
Could: Kravet kan implementeras för att om det finns tid som en utbyggnad.
Won't: Kravet ska inte implementeras för att det inte fyller ett syfte till slutprodukten eller att det inte finns tid över för att implementeras.

Beskrivning av id:t för kvalitativa krav

Första bokstaven är K och står för att det är ett kvalitativt krav. De kommande bokstäverna är för vilket område kravet tillhör och sista siffran för att numrera så de blir unika. Exempelvis K.P.1 = Kvalitativt krav inom programmering med nummer 1.

Java-projekt

K.J.1 Ethernet (Could)

Server kommunicerar med klienten med Ethernet.

K.J.2 Max antal tecken i namninput (Should)
Användaren kan välja ett namn på max 20 tecken.

Programmering

Programkoden ska följa de generella riktlinjerna för Java[2] och C++[3].
Programmeringskoden ska skrivas enligt några specifika riktlinjer.

K.P.1 Programmeringsspråk (Must)
Programmet skrivs i Java eller C++.

K.P.1.1 Klient (Must)
Koden skrivs i java

K.P.1.2 Server (Must)
Koden skrivs i java

K.P.1.3 Inbyggt System (Should)
Koden skrivs i C++

K.P.2 Variabel och funktionsnamn (Must)
Namnen skrivs i camelCase och PascalCase och är tydliga där personer som inte läst koden kommer förstå.

Systemet som helhet

K.Sy.1 Latens (Must)
Max 500ms latens i systemet fördelat på Klient, Server och Inbyggda systemet.

Maximala svarstiden från ett knapptryck till fysiskt rörelse ska vara så låg att den inte märks av användaren och leder till till någon form av köbildning.
Maximala svarstiden från ett knapptryck till fysisk rörelse ska vara 150ms.
Inbyggt system → Server → Inbyggt system → Solenoid : <300ms.
Klient → Server: <50ms.

K.Sy.2 Uptime (Should)
Systemet ska vara tillgängligt för användning 80% av tiden under demodagen

K.Sy.4.1 Omstart (Should)
Vid fel ska systemet kunna startas om på 1 minut.

[2] Java Coding Guilines
[3] C++ Coding Guilines

K.Sy.3 Skalbarhet/Flexibility(Could)
Det ska vara möjligt att lägga till fler solenoider eller utöka spelplanen utan att behöva skriva om hela systemet.

K.Sy.4 Användbarhet (Should)
Systemet ska vara enkelt att förstå efter 1 minuts introduktion för en ny användare.

K.Sy.5 Tillgänglighet (Must)
All text i systemet ska vara skrivet på Svenska eller Engelska.

Inbyggda system

K.Sy.Is.1 Half-Duplex anslutning (Could)
Arduino ska kunna skicka och ta emot värden så snabbt så att det inte märks av att koden körs på en kärna.

K.Sy.Is.2 Ventilerad design (Should)
Designen på kuberna ska vara en ventilation vänlig design som hjälper luft igenom för att minska överhettning (t.ex fins och utskärningar).

> **K.K.Is.2.1 Kylning (Must)**
> Spelet måste ha fläktar som är igång under spelets gång för att förhindra att spelet överhettas. Optimalt är temperaturer under 45 grader.

K.Sy.Is.3 Felhantering (Must)
Servern ska kunna hantera en oväntad frånkoppling av arduinon utan att krascha.

K.Sy.Is.4 Strömförsörjning (Must)
Nätaggregatet ska kunna tillföra tillräckligt med ström för att försörja spelbrädan när alla solenoider är aktiva.

K.Sy.Is.5 Prestanda (Should)
Solenoiderna bör inte tappa styrka över tid.